

GODOT
Game engine

Il Manuale

Corso di Godot 🎮

Versione 0.4 · 26/07/2026

Corso a cura del prof. Nicola Regge

Come si valutano i compiti

Le regole del gioco, chiare fin da subito.

Qui non ti freghiamo: sai **in anticipo** come funziona la valutazione. Leggila una volta, poi lavora tranquillo.

1. Il codice che funziona è il biglietto d'ingresso, non il voto. Consegnare il gioco che gira ti fa “entrare alla prova”. Ma il codice **da solo** non fa il voto: puoi copiarlo o fartelo scrivere... e allora non direbbe niente su di te.

2. Il voto nasce dalla prova dal vivo, da solo. Uno di questi due (o tutti e due):

- **Me lo spieghi:** racconti **a parole tue** cosa fa il tuo codice.
- **Il gioco rotto:** ti do il tuo gioco con **2-3 errori nascosti** dentro, e tu lo **rimetti in piedi lì per lì** — da solo, senza AI e senza chiedere ai compagni.

Se hai capito, li fai in pochi minuti. Se hai solo incollato, ti blocchi. Ecco perché **capire conviene**.

3. Il patto con l'AI (e con i compagni). Puoi usarli per **imparare**: capire un errore, farti spiegare, avere uno spunto. Non per **consegnare senza capire**. La prova del nove è sempre la stessa: **lo sai spiegare e riparare da solo?**

4. Zero vergogna. Usare gli aiuti (i 4 livelli, l'AI, un compagno) è **permesso e normale** — non è imbrogliare. Anche sbagliare è normale: **capita a tutti i programmatori**, pure ai più bravi. L'unica cosa che conta è che, alla fine, **tu abbia capito**.

Scrivere in Markdown

La tua dispensa, fatta con due segnetti.

Markdown (si dice “*marc-daun*”) è un modo per scrivere **testo normale** e aggiungere qualche **segnetto** che dice *come deve apparire* (questo è un titolo, questa parola è in grassetto, questo è un elenco...).

 **Il ponte con quello che conosci:** in Word premi il bottone grassetto; qui il bottone non c'è, il grassetto lo **scrivi tu** mettendo due asterischi attorno alla parola. Tutto qui. I segnetti li scrivi tu, ma nel risultato finale **non si vedono**.

La tabella dei segnetti

Vuoi ottenere...	Scrivi così
Un titolo grande	# Il mio titolo
Un titolo più piccolo	## Sottotitolo (più # = più piccolo)
Grassetto	**parola** (due asterischi prima e dopo)
<i>Corsivo</i>	<i>*parola*</i> (un asterisco prima e dopo)
Un punto elenco	- prima cosa (trattino + spazio)
Un elenco numerato	1. prima cosa
Un nome tecnico / codice in riga	<code>_process`</code> (un apice basso prima e dopo)
Un' immagine (tuo screenshot)	![il mio gioco](immagini/es1.png)
Una nota/riquadro	> Attenzione: ricordati di salvare!

 L'apice basso ``` (backtick) su tastiera italiana si fa con **Alt Gr** + `'` (il tasto dell'apostrofo, vicino allo `0`).

Un blocco di codice

Metti **tre apici bassi** prima e **tre dopo**: così il codice viene mostrato bello incolonnato.

```
```gdscript
func _ready():
 print("Ciao!")
```
```

Le 4 regole d'oro

1. **Riga vuota tra un paragrafo e l'altro.** Se non la metti, le frasi si attaccano tutte insieme.
2. **Uno spazio dopo # e dopo -.** `#Titolo` non funziona, `# Titolo` sì.
3. **I segnetti non si vedranno:** servono solo a dare la forma. Non spaventarti se nel testo grezzo sembrano strani.
4. **Bloccato? Fatti aiutare bene dall'AI.** Scrivi con **parole tue**, poi chiedi “*mettimi questo in Markdown*”, e **guarda come l'ha fatto** — così impari il segnetto per la prossima volta. (Ricorda il patto: l'AI ti aiuta a *formattare*, il pensiero resta tuo.)

Come parti da un modello (template)

Non parti mai da un foglio bianco: c'è un **modello** già pronto (con i titoli e i posti da riempire). Ecco come lo apri e te ne fai **una copia tua**:

1. `[BROWSER]` apri il file del **modello** nel repository del corso.
2. In alto, sopra il testo del file, clicca `Copy raw file` (l'iconcina che copia tutto il contenuto).
3. Vai nel **TUO** repository → bottone `Add file` → `Create new file`.
4. Dai un nome che finisce con `.md` (es. `es1.md`).
5. **Incolla** (`Ctrl + V`) il modello, poi **riempi i vuoti** con le tue cose.
6. Bottone verde `Commit changes` per salvare. Fatto! 

Come metto un mio screenshot

1. Fai lo screenshot del **tuo** gioco (in Windows: `Win + Shift + S`).
2. Caricalo nella cartella `immagini/` con il nome che trovi già scritto nel modello (es. `es1.png`).
3. La riga nel modello è **già pronta**: `![il mio gioco](immagini/es1.png)` → l'immagine comparirà da sola.

 **Prova del nove:** se guardi la tua pagina e vedi il titolo grande, il grassetto e il tuo screenshot al posto giusto... **ce l'hai fatta!**

Cos'è Godot (e com'è “parente” di Lazarus)

Godot è un programma gratuito per creare **giochi** e app interattive. È molto simile, come spirito, a **Lazarus**: entrambi sono ambienti gratuiti dove **componi qualcosa a schermo e ci attacchi del codice**.

La “stele di Rosetta” Lazarus → Godot:

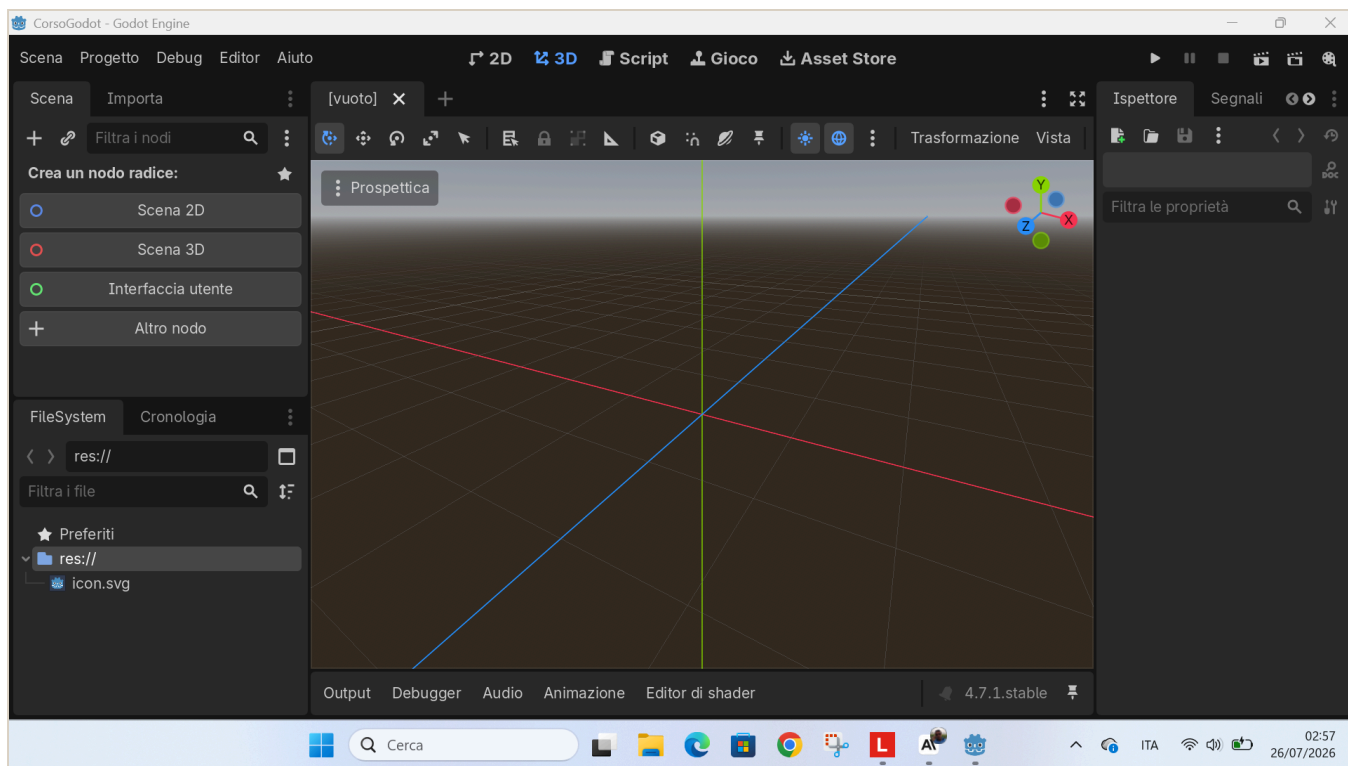
| In Lazarus (lo sai) | In Godot (nuovo) |
|---|---|
| Progetto | Progetto (uguale) |
| Form (la finestra) | Scena (un <i>albero di nodi</i> , più potente) |
| Componenti (TButton, TEdit...) | Nodi (Button, LineEdit, Label...) |
| Proprietà (Caption...) nell'Object Inspector | Proprietà nell'Ispettore |
| Gestore evento (<code>Button1Click</code>) | Segnale + funzione |
| Object Pascal (il linguaggio) | GDScript (linguaggio, stile Python) |

La differenza più importante — il “game loop”: in Lazarus il programma è **fermo** finché non clicchi qualcosa. In Godot c'è una funzione, `_process(delta)`, che **gira da sola ~60 volte al secondo**, in continuazione. È questo che fa muovere le cose (personaggi, oggetti che cadono, animazioni).

💡 In una frase: **Lazarus reagisce, Godot pulsa.**

L'ambiente di sviluppo all'avvio

Quando apri un progetto nuovo, l'editor si presenta così: vista **3D** di default e scena ancora vuota.



L'editor di Godot appena aperto, con la scena vuota

Per il nostro corso lavoreremo quasi sempre in **2D**: cliccheremo “**Scena 2D**” per iniziare (lo vediamo nel Capitolo 1).

I 4 concetti base di Godot

Se capisci questi quattro, capisci Godot:

| Concetto | Cos'è | Analogia LEGO |
|---------------------------------------|--|--------------------|
| Progetto | La cartella con dentro il file <code>project.godot</code> e tutto il gioco | La scatola del set |
| Nodo | Il mattoncino base (Sprite2D = immagine, Label = testo, Timer = cronometro...) | Un pezzo di LEGO |
| Scena | Tanti nodi messi ad albero, salvati in un file <code>.tscn</code> | Una costruzione |
| Script
(<code>.gd</code>) | Codice GDScript attaccato a un nodo, che gli dà comportamento | Le istruzioni |

Regola d'oro: in Godot **tutto è un nodo**; le scene sono nodi messi insieme; gli script danno vita ai nodi.

GDScript: il linguaggio

GDScript è la lingua che si parla **dentro** Godot. È stato fatto apposta per somigliare a **Python**: si legge facile, si usa il **rientro (TAB)** per raggruppare le righe, **niente punto e virgola**.

Due funzioni speciali che Godot chiama da solo:

- `func _ready():` → eseguita **una volta**, all'avvio (per preparare le cose).
- `func _process(delta):` → eseguita **a ogni fotogramma** (il game loop). `delta` = secondi passati dall'ultimo fotogramma; serve a muoversi in modo fluido su qualsiasi PC.

Esempio minimo (leggere le frecce e spostare qualcosa):

```
func _process(delta):  
    if Input.is_action_pressed("ui_left"):  
        posizione.x -= 200 * delta    # vai a sinistra  
    if Input.is_action_pressed("ui_right"):  
        posizione.x += 200 * delta    # vai a destra
```

Il nostro primo gioco: “Chirurgo Pasticcione”

Idea: un chirurgo maldestro fa cadere gli organi dal tavolo. Tu muovi il **vassoio** con le frecce $\leftarrow$ $\rightarrow$ e li prendi al volo.

- organo preso $\rightarrow$ **+1 punto**
- organo per terra $\rightarrow$ **-1 vita**
- zero vite $\rightarrow$ **Operazione Fallita** (INVIO per riprovare)

Cosa ci insegna:

- Creare nodi da codice (il vassoio, gli organi, le scritte).
- Il **movimento** con le frecce (input + `delta`).
- Le **collisioni** (quando il vassoio “tocca” un organo).
- Tenere lo **stato del gioco** (punti, vite) e mostrarlo a schermo.
- Un **Timer** che fa comparire gli organi a intervalli.

Il codice completo e commentato è in `godot/chirurgo-pasticcione/main.gd`.

Il percorso: dagli esercizi al “progetto boss”

Qui non si impara con la teoria astratta, ma **facendo**. Ogni esercizio insegna **un pezzo**; poi arriva un gioco più grande — il “**progetto boss**” — che mette insieme quei pezzi. Ecco la scala che stiamo salendo.

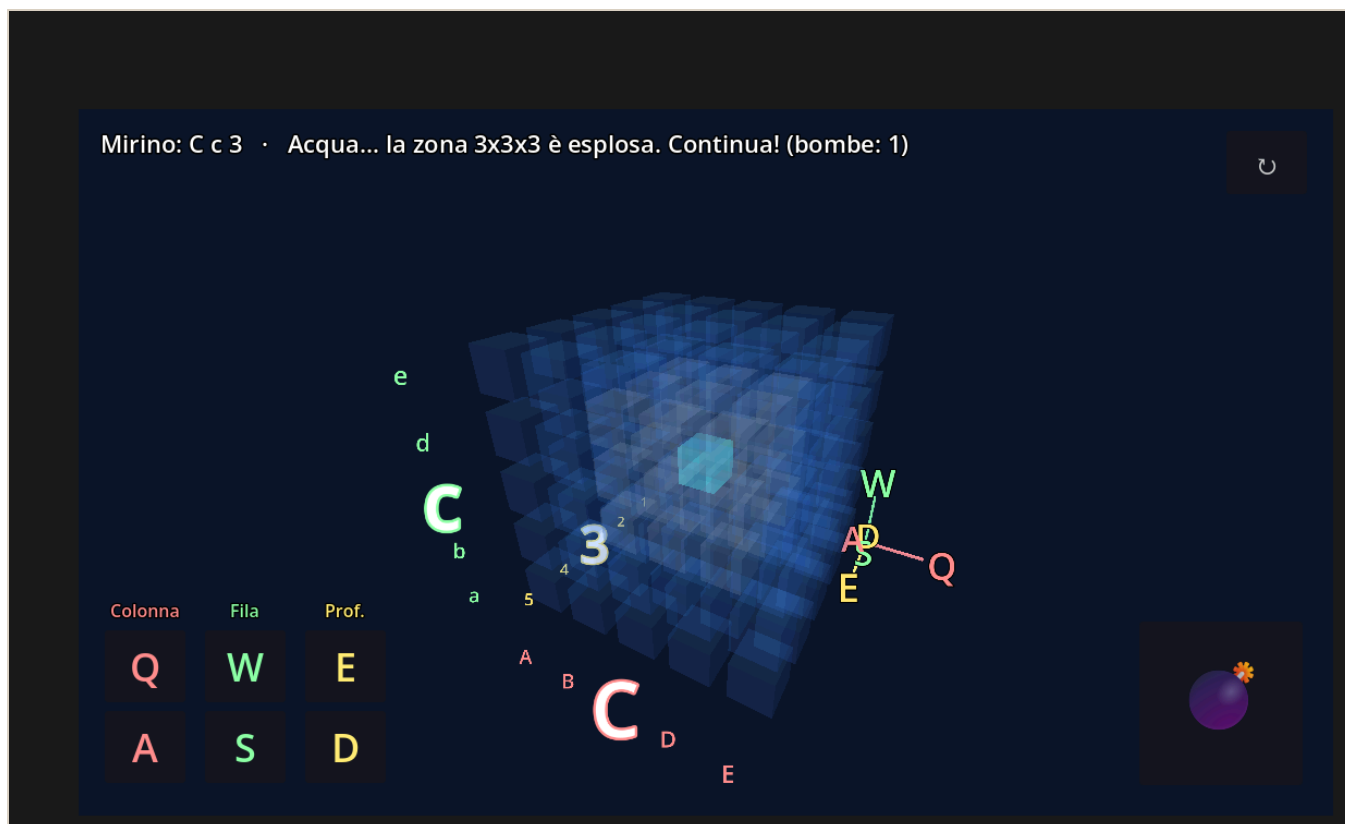
I gradini piccoli (gli esercizi dell’eserciziario)

| Esercizio | Cosa impari | Il concetto sotto |
|---------------------------|-------------------------------|---|
| 1 • Il bottone che saluta | Un clic fa succedere qualcosa | Il segnale (è il tuo <code>Button1Click</code> di Lazarus) |
| 2 • Muovi il quadrato | Far muovere le cose da sole | Il game loop <code>_process(delta)</code> + input |
| 3 • Prendi la moneta | Un mini-gioco vero | Movimento + collisioni + punteggio |

Ognuno è **corto** e finisce con una **vittoria a schermo**: è fatto apposta così, per vincere subito e non mollare.

Cos’è un “progetto boss”

È un gioco **già pronto**, più grosso, che **non si copia riga per riga**. Si **apre, si gioca e si rende proprio** (cambi i colori, il titolo, ci metti una tua foto). È il **premio**: la cosa figa da mostrare subito. Il primo è “**Affonda la Bonomi**” (lo trovi nell’eserciziario e nella cartella `battaglia-navale-3d/`).



Il "progetto boss" Affonda la Bonomi: una battaglia navale in 3D, dentro un cubo d'acqua.

Dal 2D al 3D (cosa cambia nel boss)

Finora abbiamo lavorato in **2D**: due coordinate, **x** (orizzontale) e **y** (verticale) — un `Vector2`. Il boss è in **3D**: si aggiunge una terza coordinata, **z** (la **profondità**) — un `Vector3`. Il campo di gioco non è più una griglia piatta ma un **cubo** di celle.

Due idee nuove, ma **niente panico**:

- **Si costruisce tutto da codice** (le celle, le luci, la telecamera, i bottoni) invece che a mano nell'editor: sono sempre gli stessi **nodi**, solo tanti, creati con un ciclo `for`.
- Sono gli **stessi concetti di prima, in grande**: il **game loop** gira il cubo, l'**input** muove il mirino. Chi ha fatto gli esercizi 2 e 3 ha già visto tutto.

Come lo proponiamo ai ragazzi (la parte che riescono a fare)

La regola d'oro: **ognuno deve poter salire almeno un gradino** e portarsi a casa una vittoria vera. Non si "finisce" il boss tutto in una volta: ci si torna più volte durante l'anno, un gradino per volta.

1. ● **Gioca** e scegli la difficoltà (4 = facilissimo). → *ci riescono tutti*.
2. ● **Cambia un colore** dell'acqua o del mirino (basta cambiare un numero nel codice). → facile, effetto immediato.

3. 🟡 **Metti la tua foto** (o un meme) al posto di quella di default.
4. 🟡 **Cambia il titolo** del gioco.
5. 🟠 **Leggi una funzione piccola** e spiega **a voce** cosa fa (es. come si muove il mirino). → è la “prova del nove”.
6. 🔴 **Per i più veloci:** cambia la potenza della bomba o aggiungi un secondo sottomarino.

Così **nessuno resta fuori**: chi è più indietro gioca e cambia un colore (è già una vittoria mostrabile), chi corre di più mette le mani nel codice. Vale sempre: **Vinci subito · Fallo tuo · Mostralo.**

Come useremo l'AI (regola per i ragazzi)

L'AI è come la **calcolatrice in matematica**: aiuta, ma se non capisci cosa stai facendo non serve a niente.

-  Usala per: capire un errore, farti spiegare un concetto, avere un suggerimento, uno **spunto da studiare e modificare**.
 -  Non usarla per: farti scrivere tutto e consegnarlo senza capirlo.
 - **Prova del nove**: se sai **spiegare a voce, riga per riga**, il codice che presenti, la competenza c'è.
-

Changelog del manuale

| Versione | Data | Cosa è cambiato |
|----------|------------|---|
| 0.1 | 26/07/2026 | Prima stesura: Cap. 0 (Godot vs Lazarus), Cap. 1 (i 4 concetti), Cap. 2 (GDScript, game loop), Cap. 3 (Chirurgo Pasticcione), regola uso AI. |
| 0.2 | 26/07/2026 | Aggiunte due schede iniziali: Scheda 1 “Come si valutano i compiti” e Scheda 2 “Scrivere in Markdown” (con la tabella dei segnetti e come partire da un modello). |
| 0.3 | 26/07/2026 | Aggiunto il Capitolo 4 “Il percorso: dagli esercizi al progetto boss”: collega i 3 esercizi ai concetti, spiega cos’è un progetto boss e il passaggio 2D → 3D, e come proporre “Affonda la Bonomi” ai ragazzi a gradini (con screenshot). |
| 0.4 | 26/07/2026 | Stile: sottotitoli delle schede/capitoli resi come sottotitolo centrato più piccolo; blocchi di codice nero-su-bianco (fondo chiaro) per stampare senza sprecare toner. |